

SOLID PRINCIPLES

Practical Assignment — Order Processing Domain

Time Allowed: 30 Minutes
Difficulty: Intermediate
Focus: Identify & Refactor
Principles: SRP · OCP · LSP · ISP · DIP

You are given one messy class (`OrderProcessor_Messy.cs`) that violates ALL 5 SOLID principles.

Your mission:

- Identify EXACTLY where each violation occurs in the code.
- Refactor the code to fix each violation.
- End with multiple small, focused classes — no business logic deleted.

Rule: You may NOT delete any business logic. Only reorganize it.

Violations briefly

SRP `OrderProcessor` does 7 things: validate, discount, save, email, report, export, log

OCP `GetDiscount()` uses an if/else chain — adding a new type requires editing the method

LSP `ArchiveOrderStorage` extends `SqlOrderStorage` but throws `NotSupportedException` on `Save()`

ISP `IOrderService` has 4 unrelated methods — classes are forced to implement what they don't need

DIP `OrderProcessor` uses new `SqlOrderStorage()`, new `SmtEmailSender()`, new `FileOrderLogger()`

The Messy Class — Find the Violations

Study the code below. Annotations in the comments point to violations. Identify the root cause of each.

```
namespace OrderSystem;

// — Entities —
public class Order
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string CustomerEmail { get; set; }
    public string OrderType { get; set; } // "Standard", "Premium", "Bulk"
    public decimal TotalAmount { get; set; }
    public List<OrderItem> Items { get; set; } = new();
}

public class OrderItem
{
    public string ProductName { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}

// — [VIOLATION 1: ISP] One fat interface forces ALL methods on every implementor
public interface IOrderService
{
    void ProcessOrder(Order order); // processing concern
    void SendConfirmationEmail(Order order); // notification concern
    string GenerateReport(IEnumerable<Order>); // reporting concern
    string ExportToCsv(IEnumerable<Order>); // export concern
}

// — [VIOLATION 2: SRP + DIP] God class with 5 responsibilities
public class OrderProcessor : IOrderService
{
    // [VIOLATION 3: DIP] Hardcoded concrete dependencies
    private readonly SqlOrderStorage _storage = new SqlOrderStorage();
    private readonly SmtpEmailSender _emailer = new SmtpEmailSender();
    private readonly FileOrderLogger _logger = new FileOrderLogger();

    public void ProcessOrder(Order order)
    {
        _logger.Log($"Processing order {order.Id}");

        // Responsibility: Validation (should be a separate class)
        if (order.Items.Count == 0) { _logger.Log("No items."); return; }
        if (string.IsNullOrEmpty(order.CustomerEmail)) { return; }

        // Responsibility: Discount calculation
        var discount = GetDiscount(order);
        var finalAmount = order.TotalAmount - (order.TotalAmount * discount);

        // Responsibility: Persist
        _storage.Save(order);
    }
}
```

```
// Responsibility: Notify
SendConfirmationEmail(order);
}

// [VIOLATION 4: OCP] Every new order type = edit this method
private decimal GetDiscount(Order order)
{
    if (order.OrderType == "Standard") return 0.00m; // edit here for new
types
    else if (order.OrderType == "Premium") return 0.10m;
    else if (order.OrderType == "Bulk") return 0.20m;
    else return 0.00m;
}

public void SendConfirmationEmail(Order order) =>
    _emailer.Send(order.CustomerEmail, $"Order {order.Id} Confirmed", "...");

public string GenerateReport(IEnumerable<Order> orders) =>
    $"Orders: {orders.Count()} | Revenue: {orders.Sum(o => o.TotalAmount):C}";

public string ExportToCsv(IEnumerable<Order> orders) =>
    string.Join("\n", orders.Select(o =>
    $" {o.Id},{o.CustomerEmail},{o.TotalAmount}"));
}

// — [VIOLATION 5: LSP] ArchiveOrderStorage breaks base class contract
public class SqlOrderStorage
{
    public virtual void Save(Order o) => Console.WriteLine($"[SQL] Saved {o.Id}");
    public virtual IEnumerable<Order> GetAll() => Enumerable.Empty<Order>();
}

public class ArchiveOrderStorage : SqlOrderStorage // read-only, yet inherits
Save()
{
    public override void Save(Order order) // BREAKS the contract!
        => throw new NotSupportedException("Read-only archive — Save() not
supported.");

    public override IEnumerable<Order> GetAll() =>
        Enumerable.Empty<Order>(); // simulates fetching from archive
}

// — Concrete infrastructure classes —————
public class SmtEmailSender { public void Send(string to, string sub, string body)
=> Console.WriteLine($"[SMTP] {to}"); }
public class FileOrderLogger { public void Log(string msg) =>
Console.WriteLine($"[LOG] {msg}"); }
```

Assignment Tasks

Complete each task in order. Each task builds refactoring skills progressively.

#1 Single Responsibility Principle (SRP)

🕒 5 min

The Problem:

OrderProcessor currently handles: validation, discount calculation, persistence, email notification, report generation, and CSV export — all in one class.

Your Task:

Split OrderProcessor into separate classes, each with ONE reason to change.

Hint: Ask yourself: "Why would this code change?" If you can give two different reasons — it belongs in two different classes.

Expected Output:

- OrderValidator — validates orders only
- OrderStorage — persists orders only
- OrderEmailSender — sends emails only
- OrderReportService — generates reports and exports
- OrderProcessor — orchestrates: validate → discount → save → notify

#2 Open/Closed Principle (OCP)

🕒 5 min

The Problem:

The GetDiscount() method uses an if/else chain on the OrderType string. Adding a "VIP" order type means editing existing code — directly violating OCP.

Your Task:

Replace the if/else chain with the Strategy Pattern — adding a new discount type = adding a new class only.

Hint: Create IDiscountStrategy with one method: decimal GetDiscount(Order order); Then implement one class per order type.

Expected Output:

- IDiscountStrategy — the closed contract
- StandardDiscountStrategy — returns 0%
- PremiumDiscountStrategy — returns 10%
- BulkDiscountStrategy — returns 20%
- OrderProcessor — receives IDiscountStrategy, never changes for new types

#3 Liskov Substitution Principle (LSP)

🕒 5 min

The Problem:

ArchiveOrderStorage extends SqlOrderStorage but throws NotSupportedException on Save(). Any caller using SqlOrderStorage will crash if ArchiveOrderStorage is substituted — breaking the base class contract.

Your Task:

Fix the hierarchy so every implementation can be safely substituted without unexpected exceptions.

Hint: Ask — "Does ArchiveOrderStorage share the SAME contract as SqlOrderStorage?" No — it can only READ. Solution: split into two focused interfaces.

Expected Output:

- IOrderWriter — contract: void Save(Order order)
- IOrderReader — contract: IEnumerable<Order> GetAll()
- SqlOrderStorage — implements both IOrderWriter + IOrderReader
- ArchiveOrderStorage — implements IOrderReader ONLY (no forced Save)

#4 Interface Segregation Principle (ISP) 5 min**The Problem:**

IOrderService has 4 methods across different concerns. A class that only sends emails is still forced to implement ProcessOrder, GenerateReport, and ExportToCsv.

Your Task:

Split IOrderService into smaller, focused interfaces. Group methods by "who uses this?"

Hint: Processing logic → IOrderProcessor | Notification → IOrderNotifier | Reporting → IOrderReporter

Expected Output:

- IOrderProcessor — ProcessOrder(Order order)
- IOrderNotifier — SendConfirmationEmail(Order order)
- IOrderReporter — GenerateReport(...) + ExportToCsv(...)
- Each implementation only implements the interface it actually needs

#5 Dependency Inversion Principle (DIP) 10 min**The Problem:**

OrderProcessor uses new() to instantiate SqlOrderStorage, SmtplibEmailSender, and FileOrderLogger directly. You cannot swap implementations or write unit tests without a real DB and mail server.

Your Task (3 Parts):**Part A — Apply DIP:**

Create IOrderStorage, IOrderEmailSender, and IOrderLogger interfaces. Replace all concrete usages with these abstractions.

Part B — Apply Constructor Injection:

Accept all dependencies via the constructor. OrderProcessor must have zero new() calls for dependencies.

Part C — Manual Wiring (IoC Container simulation):

In a static Setup() method, manually wire all dependencies together — simulating what an IoC container does.

Key Reminder: DIP ≠ DI — DIP is the rule (depend on interfaces). DI is the technique (push them in from outside). Both are required here.

Expected Output:

- IOrderStorage — abstracts where orders are saved
- IOrderEmailSender — abstracts how emails are sent
- IOrderLogger — abstracts where logs go
- SqlOrderStorage — implements IOrderStorage
- SmtEmailSender — implements IOrderEmailSender
- ConsoleOrderLogger — implements IOrderLogger
- OrderProcessor — receives all 3 via constructor, zero new() calls for dependencies

BONUS — Full Refactor

Combine ALL your fixes into a single, clean design where:

- Every class has ONE reason to change
 - SRP — focused, cohesive classes
- Adding a new discount = one new class only
 - OCP — extend via new code, not edits
- Every implementation is safely substitutable
 - LSP — subclasses honor their contracts
- No class implements methods it doesn't use
 - ISP — small, focused interfaces
- OrderProcessor depends only on abstractions
 - DIP — constructor injection, no new() for deps

Use any remaining time to verify your design is testable: could you swap `SqlOrderStorage` for an in-memory mock without touching `OrderProcessor`?

Time Budget Guide

#	Principle	Time	Priority
1	Single Responsibility Principle	5 min	Core
2	Open/Closed Principle	5 min	Core
3	Liskov Substitution Principle	5 min	Core
4	Interface Segregation Principle	5 min	Core
5	Dependency Inversion Principle	10 min	Hardest
TOTAL		30 min	

Evaluation Criteria

Criteria	Points	Notes
Correctly identifies each violation location	10	2 pts each
Correct SRP refactor — focused, single-purpose classes	10	Task 1
OCP fix uses Strategy Pattern correctly	10	Task 2
LSP fix — no runtime exceptions on substitution	10	Task 3
ISP — 3 focused interfaces, no forced methods	10	Task 4
DIP — constructor injection, zero new() for deps	20	Task 5 (A+B+C)
No business logic deleted (reorganized only)	10	All tasks
Bonus: Full combined refactor compiles cleanly	20	Bonus